

Iceberg runtime pruning in DataFusion Comet — benchmark summary

Prepared 2026-10-09 · fork [unikdahal/datafusion-comet](#) + [unikdahal/iceberg-rust](#)

Apache Spark 3.5 + Apache Iceberg 1.8.1 · Comet native Iceberg scan · compared against **unmodified Apache DataFusion Comet main** (freshly resolved per campaign) · all runs on GitHub-hosted 4-core runners

0

wrong results from the candidate across 4,403 queries (403 timed + 4,000 fuzz), every result checked against plain Spark

3.3–3.6×

geometric-mean speedup on the join and Top-K / MIN-MAX suites (latest full campaign)

up to 49×

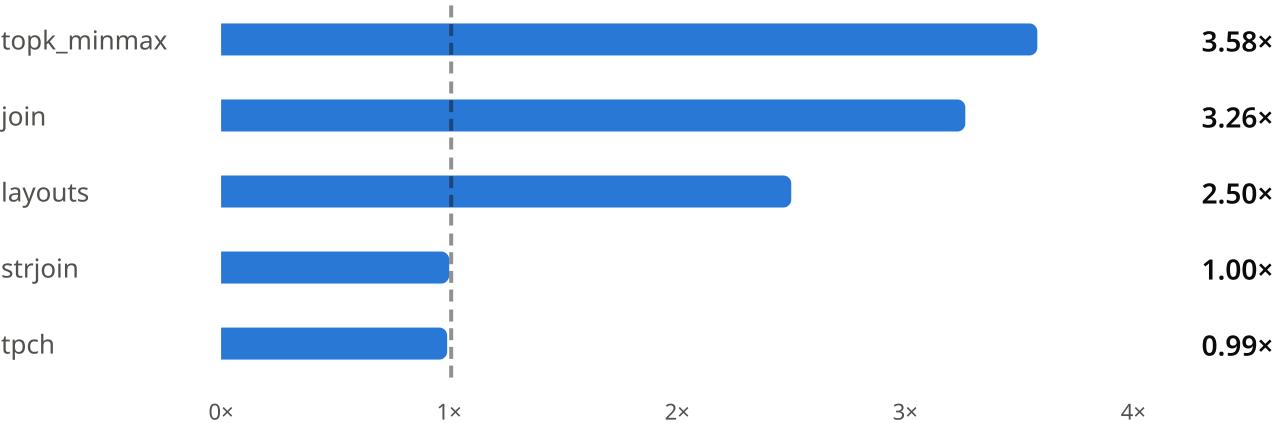
best single query (equality-delete scan on string keys)

≈1.00×

TPC-H and non-selective string joins — no measurable cost where pruning cannot help

1. Latest full campaign — speedup vs Apache Comet main

Campaign run 37788815352 · candidate Comet 434d18333 + iceberg-rust 1e3f2a868 · baseline Apache main 5db0efe69. Speedup = baseline median time ÷ candidate median time, runtime pruning enabled on both sides. Geometric mean over queries where both engines ran the same native operators (“equal coverage”).



Dashed line = 1.0× (parity).

Suite	What it exercises	Queries	Geomean (equal coverage)	Geomean (all)	Faster (95% CI > 1)	Below parity (CI < 1)	Best query
topk_minmax	ORDER BY ... LIMIT, MIN/MAX, deletes	85	3.58×	3.78×	65	1	49.2×
join	selective broadcast / shuffled / semi joins, deletes	59	3.26×	3.26×	39	4	39.6×

Suite	What it exercises	Queries	Geomean (equal coverage)	Geomean (all)	Faster (95% CI > 1)	Below parity (CI < 1)	Best query
layouts	sorted / skewed / evolved / many- file tables	89	2.50×	2.65×	68	3	18.3×
strjoin	large, mostly non- selective joins (int/long/string keys)	121	1.00×	1.00×	6	9	1.17×
tpch	TPC-H, natural and clustered layouts	44	0.99×	0.99×	2	5	1.05×
Total		398			180	22	

“Faster / below parity” use each query’s bootstrap 95% confidence interval over 8 balanced rounds. 5 more queries are excluded because Apache main returned wrong results (see §3). Unequal coverage: 25 queries where Apache main fell back to Spark’s JVM scan (tables whose names end in files/snapshots are mistaken for Iceberg metadata tables on main); they are excluded from the equal-coverage geomean.

Largest single-query speedups

Query	Suite	Speedup [95% CI]	Why
equality_delete_scan__f_eq_deletes_str	topk_minmax	49.2× [46.2, 52.5]	equality deletes compacted into a membership predicate
equality_delete_scan__f_eq_deletes_str	join	39.6× [39.2, 40.1]	same, inside a join
max / min__f_eq_deletes_str	topk_minmax	36.2× / 35.7×	runtime MIN/MAX bound + delete handling
topk10__f_eq_deletes_str	topk_minmax	33.9× [32.0, 36.2]	Top-K threshold prunes files and row groups
join_inner__f_eq_deletes_str_dim_128	join	22.8× [21.2, 24.5]	join keys prune the fact table at read time
join_inner__f_big_dim_10k	layouts	18.3× [17.6, 19.0]	runtime join filter skips most of a large table

2. Features added after that campaign (targeted benchmarks)

Measured with targeted runs (same harness, same methodology, subset of queries). Not yet part of a full campaign.

Capability	Query	Before	After	Data read	Run
Decimal join keys	join_inner__f_dec__dim_10k	0.96×	5.19×	589 → 13 MiB	37829425897
Decimal join keys	join_inner__f_dec__dim_128	0.97×	4.52×	589 → 12 MiB	37829425897
String join keys	join_inner__f_str__dim_128	≈1.0×	4.79×	574 → 13 MiB	37865998293
String join keys	join_inner__f_str__dim_10k	≈1.0×	4.20×		37865998293
String keys + deletes	join_inner__f_eq_deletes_str__dim_128		13.7×		37865998293
Adaptive filter bypass	non-selective 3M-key joins	eval 15–33 ms	eval 3–8 ms	selective joins kept full pruning	37807233835

Also implemented and CI-validated, benchmark pending: string Top-K / MIN-MAX pruning, file-level pruning for decimal/string keys (today 0 of 22 files skipped even though 80 of 80 row groups are), NULLS FIRST Top-K, filters through safe intermediate operators, a driver-side statistics cache (removes up to ~24 ms of planning on tables with many manifests).

3. Correctness

- ✓ Candidate: **0 result mismatches** in every campaign — 403 timed queries × 4 variants × 8 rounds, plus 4,000 randomized fuzz queries per campaign (NaN, ±0, nulls, deletes, schema evolution).
- ▲ Apache main (baseline) returned **wrong results** on broadcast joins with string keys built from a DISTINCT (sliced Arrow buffers corrupted during broadcast). Reproduced in an isolated test, root-caused, fixed in the fork, and filed as fork issue #42. Those queries are excluded from timing comparisons.
- ✓ Defects found and fixed in our own branch before release: NaN / signed-zero handling in a row-filter shortcut, delete-file loading continuing after cancellation, an over-eager shuffle-probe optimisation (reverted), a row-filter gating change that regressed selective queries (reverted after measurement).

4. Where we are not faster (and why)

- TPC-H (0.99×)**: join keys are not correlated with the physical sort order, so there is nothing to prune; the result shows the pruning machinery costs ≈ nothing when it cannot help.
- Large non-selective joins (strjoin 1.00×)**: filters remove few rows; an adaptive bypass now switches filtering off when it is not paying for itself.

- **22 queries with a confidence interval below 1.0×** in this single campaign (0.90–0.98×, mostly sub-100 ms queries). None reached the harness’s 0.90× regression threshold. Policy: a regression counts only if it reproduces in two independent campaigns — several earlier “regressions” disappeared on re-measurement (e.g. 0.88× → 1.03×).

5. Methodology

Variants	baseline (Apache main) and candidate (fork), each with runtime pruning on and off — 4 variants per query
Repetition	8 balanced JVM rounds per variant (order rotated), 4 warm-ups per query, raw per-run timings kept
Statistics	medians; bootstrap 95% confidence intervals on the speedup ratio; suite geomeans
Fairness	same data files (byte-identical), same queries, same Spark configuration; baseline and candidate always run on the same machine for a given query; per-query operator coverage compared
Correctness gate	every result compared with plain Spark; mismatching comparisons are excluded from speedups
Infrastructure	GitHub Actions, 18 parallel shards; campaign wall time cut from ~4 h to 1 h 46 min with identical protocol
Caveats	shards run on different CPU models (AMD EPYC 7763 / 9V45 / 9V74, Intel Xeon 8370C) — ratios within a shard are paired, cross-campaign ratios of very fast queries vary with CPU; compare bytes/rows read for those